

Projet individuel : Algèbre Linéaire Numérique Appliquée à l'Analyse de Données

Introduction

Ce projet individuel vise à vous familiariser avec les concepts d'algèbre linéaire numérique en Python, en utilisant les bibliothèques `numpy` et `scipy`. Vous explorerez la résolution de systèmes linéaires, les décompositions matricielles (SVD et QR), et leurs applications à l'analyse de données.

Consignes

- Travail **individuel**.
- Le code Python doit être clair, commenté et structuré.

Livrables

- Un notebook Python (votrenom.ipynb)
- Un rapport PDF exporté du notebook (avec visualisations et commentaires)
- Date limite : 01 Juin 2026.

1 Exercice 1 : Résolution de Systèmes Linéaires

1.1 Méthodes Directes : Décomposition LU

Objectif : Résoudre un système linéaire $Ax = b$ en utilisant la décomposition LU.

1. Créez une matrice A (4x4) symétrique définie positive et un vecteur b (4x1) aléatoires.
2. Utilisez `scipy.linalg.lu` pour obtenir les matrices P, L, U telles que $PA = LU$.
3. Résolvez le système $Ax = b$ en utilisant les matrices L et U (résolution triangulaire).
4. Vérifiez votre solution en comparant avec `numpy.linalg.solve`.

1.2 Méthode de Cholesky

Objectif : Résoudre le même système avec la décomposition de Cholesky.

1. Utilisez `scipy.linalg.cho_factor` et `scipy.linalg.cho_solve` pour résoudre $Ax = b$.
2. Comparez la précision et le temps d'exécution avec la méthode LU.

1.3 Méthodes Itératives : Jacobi et Gauss-Seidel

Objectif : Implémenter les méthodes itératives de Jacobi et Gauss-Seidel.

1. Écrivez une fonction pour la méthode de Jacobi :
 - Décomposez $A = D + (L + U)$, où D est la diagonale, L la partie triangulaire inférieure, et U la partie triangulaire supérieure.
 - Itérez selon $x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)})$.
2. Écrivez une fonction pour la méthode de Gauss-Seidel.
3. Testez les deux méthodes sur le système $Ax = b$ avec une tolérance de 10^{-6} et un maximum de 100 itérations.
4. Comparez le nombre d'itérations et la précision.

2 Exercice 2 : Décompositions SVD et QR

2.1 Décomposition SVD

Objectif : Appliquer la SVD à une matrice de données.

1. Créez une matrice de données D (10x5) simulant un ensemble de données (par exemple, notes d'étudiants pour 5 matières).
2. Utilisez `numpy.linalg.svd` pour obtenir U, S, V .
3. Reconstituez la matrice à partir de U, S, V et vérifiez l'exactitude.
4. Réduisez la dimension en ne gardant que les 2 premières valeurs singulières et reconstituez une approximation de D .

2.2 Décomposition QR

Objectif : Appliquer la décomposition QR à une matrice.

1. Utilisez `scipy.linalg.qr` pour décomposer la matrice D en Q et R .
2. Vérifiez que Q est orthogonale ($Q^T Q = I$) et que $QR = D$.

3 Exercice 3 : Application à l'Analyse de Données

3.1 Analyse en Composantes Principales (ACP)

Objectif : Appliquer l'ACP à l'ensemble de données D (même matrice de l'exercice 2).

1. Centrez les données en soustrayant la moyenne de chaque colonne.
2. Appliquez la SVD sur la matrice centrée.
3. Projetez les données sur les deux premières composantes principales.
4. Visualisez les données projetées avec `matplotlib`.

3.2 Compression d'Image

Objectif : Appliquer la SVD pour compresser une image.

1. Chargez une image en niveaux de gris avec `matplotlib.image`.
2. Appliquez la SVD sur la matrice de l'image.
3. Reconstituez l'image avec différents rangs (par exemple, 10, 50, 100).
4. Visualisez les résultats.

4 Questions

1. Comparez les performances (temps, précision) des méthodes directes et itératives pour résoudre $Ax = b$.
2. Expliquez l'impact de la réduction de dimension via SVD sur l'analyse de données.
3. Discutez des avantages et limites de l'ACP pour l'analyse de données.

Conseils

- **Exercice 1** : Les méthodes directes (LU, Cholesky) sont efficaces pour des matrices denses de petite taille, tandis que les méthodes itératives (Jacobi, Gauss-Seidel) sont adaptées aux matrices creuses ou de grande taille. Vérifiez la convergence des méthodes itératives (par exemple, dominance diagonale pour Jacobi).
- **Exercice 2** : La SVD est puissante pour la réduction de dimension et la compression, tandis que la QR est utile pour les problèmes de moindres carrés et la stabilisation numérique.
- **Exercice 3** : L'ACP est une application classique de la SVD pour visualiser et réduire la dimension des données. Pour la compression d'image, expérimentez avec différents rangs pour observer le compromis qualité-taille.
- **Ressources** : Consultez la documentation de `numpy.linalg` et `scipy.linalg`. Les tutoriels en ligne sur l'ACP et la SVD peuvent être utiles.

Évaluation (Critères)

- Qualité du code et des visualisations.
- Profondeur de l'analyse (interprétation des composantes).
- Clarté du rapport et de la présentation.